AI-first Software Engineering Program - Case Study

Prism: LLM Gateway and Semantic Cache

Author : **Airtribe**

BACKGROUND

Every AI product used today sits behind the same invisible layer: a gateway between the application and the model providers. It's the infrastructure layer every AI company now runs, one place that handles keys, budgets, failover, caching, and cost accounting, so many teams don't each wire up providers their own way. Products like LiteLLM, OpenRouter, and Portkey exist because this is now core infrastructure, the way API gateways and load balancers were a decade ago.

Prism is an AI-first product where the gateway itself makes AI decisions in the hot path: it judges how hard each prompt is to route it to the right model tier, and it matches meaning - not strings, to serve repeated questions from cache. There's no chatbot to build here; the challenge is streaming, metering, reliability, and multi-tenancy done properly, with machine judgment running inside the request path

OBJECTIVE

Build a production-grade LLM gateway that can:

- Route requests across multiple model providers through one unified, OpenAI-compatible API.
- Judge prompt difficulty and route automatically - cheap models for simple prompts, strong models for hard ones proven against a labeled eval set.
- Fall back automatically when a provider goes down, rate-limits, or degrades.
- Stream responses token by token, end to end, without buffering.
- Meter usage per tenant - every request, every token, every dollar.
- Enforce cost budgets and rate limits per team.
- Answer repeated questions from a semantic cache instead of paying for the same tokens twice.
- Keep a request log complete enough to debug any AI traffic issue.

KEY FEATURES

Load the Provided Data

- Load the model price table, seed tenants, and gateway config (model_pricing.json, seed_keys.json, gateway_config.sample.json).
- Preserve seed key values and alias names - the demo flow and scripts reference them.

Unified Chat Completions API

- One OpenAI-compatible POST /v1/chat/completions endpoint in front of at least 2 providers (mocks count).
- Callers use a model alias/name; the gateway resolves it and never exposes provider credentials.
- Every response carries the header contract: x-prism-provider, x-prism-cache, x-prism-fallback, x-prism-cost-usd.

Streaming Pass-Through

- "stream": true flows tokens from provider → gateway → client as they arrive, no buffering.
- Stream terminates correctly (data: [DONE]); usage/cost still recorded.

Virtual Keys, Rate Limits & Budgets

- Authenticate every request with a virtual key.
- Enforce per-key model allowlist, requests-per-minute limit, and monthly cost budget.
- Distinct, documented error per rejection type (invalid key, model not allowed, rate limited, budget exhausted).

Routing, Retries & Failover

- Resolve aliases (fast, smart) to a primary model with an ordered fallback chain.
- Retry transient errors with exponential backoff; fail over and record it.

Smart Routing - the auto Alias

- "model": "auto" lets the gateway classify prompt difficulty and route to fast or smart.
- Must beat a length-only baseline - the eval set deliberately includes short-but-hard and long-but-trivial traps.
- One-command routing eval reporting accuracy per case.

Semantic Response Cache

- Serve cached responses for semantically similar prompts (not just exact matches), scoped per key.
- Report hits via the header contract and track hit rates.

Metering, Request Logs & Usage API

- Compute cost per request from the price table using provider-reported token usage.
- Log every request (key, model, provider, status, tokens, cost, cache result, fallback flag, latency) and expose a usage summary API.

Simple Ops Console & Verification

- A simple frontend: usage by key, recent requests, cache hit rate (vibe-coding fine).
- Must pass scripts/smoke_test.py; run scripts/load_test.py and the routing eval, and report results.

TECHNICAL REQUIREMENTS

- Expose core functionality as an HTTP API. The data plane must remain OpenAI-compatible.
- Use a reliable persistent store for virtual keys, usage records, request logs, and cache entries.
- Usage accounting and rate limiting must work correctly under concurrent requests.
- Every upstream provider call must have a timeout so the gateway does not hang when a provider is slow or dead.
- Keep provider integration behind an adapter so mock providers or real providers can be used without changing the core system.
- Provide automated tests for critical logic such as cost computation, rate limiting, cache decisions, allowlists, and routing behavior.

ASSESSMENT CRITERIA

- Functionality:
  Does the gateway load the provided data, serve OpenAI-compatible completions over two providers, stream responses correctly, enforce keys, rate limits and budgets, perform failover, route auto traffic, cache semantically, meter usage, and show platform activity through a simple console?
- Gateway Correctness Under Pressure:
  Does usage accounting remain accurate during concurrent requests? Does the load test show no over-admission? Does provider failover work when a provider is taken down? Does the cache hit paraphrased prompts without leaking responses across tenants?
- Code Quality:
  Is the code clean, organized, maintainable, and easy to understand?
- Design and Structure:
  Is the gateway well-designed? Does it show a strong understanding of backend architecture, streaming, provider adapters, rate limiting, cost metering, retries, failover, and multi-tenant systems?
- Documentation and Demo:
  Is the README clear? Are setup steps, APIs, design decisions, cache scoping, routing logic, failover behavior, and known limitations documented? Does the demo clearly show the full Prism workflow?

DELIVERABLES

- The final, functional product: LLM gateway and simple ops console.
- README file covering setup instructions, API documentation, architecture, routing/failover design, cache scoping, and design decisions.
- Public link of the GitHub repository.
- Seeded demo keys and instructions to reproduce the demo with mock providers.
- Verification report including smoke test output, load test report, routing eval accuracy, over-admission results, accounting totals, and latency summary.
- Explainer video demonstrating the project, including streaming, smart routing, budget rejection, rate limiting, semantic cache hit on a paraphrased prompt, and live provider failover.

RESOURCES

GitHub Repository:
https://github.com/airtribe-projects/prism-capstone